FoodSeg103 Dataset Rebalancing Notebook

Create a new dataset version by:

1. Removing low-frequency classes (configurable threshold).
2. Merging semantically similar classes (custom mapping).

This notebook is designed to mitigate class imbalance before training.

Workflow

1. Configure thresholds and merge rules.
2. Scan annotation files and compute class statistics.
3. Build a class remapping plan.
4. Export a new filtered and merged dataset.

In [18]:
```
import json
import shutil
from collections import Counter, defaultdict
from difflib import SequenceMatcher
from pathlib import Path

import pandas as pd
from tqdm.auto import tqdm
```

```
# ==========================
# User Configuration
# ==========================
CANDIDATE_ROOTS = [
    Path("./foodseg103"),
    Path("./test/foodseg103"),
    Path("../foodseg103"),
]

OUTPUT_ROOT = Path("./foodseg103_rebalanced")
SPLITS = ["train", "test"]
IMAGE_EXTS = (".jpg", ".jpeg", ".png")

# Keep class only if merged class satisfies both
MIN_IMAGES_PER_CLASS = 0
MIN_OBJECTS_PER_CLASS = 0

# Optional explicit overrides.
FORCE_KEEP_CLASSES = set([
    # "rice",
    "king oyster mushroom",
    "oyster mushroom",
    "enoki mushroom",
])
FORCE_DROP_CLASSES = set([
    # "unknown",
    "kelp",
    "bamboo shoot",
    "egg tart"
])

# Mapping from old class name to new merged clas
# Edit this to merge similar classes.
MERGE_MAP = {
    # "chicken duck": "poultry",
    # "duck": "poultry",
    # "fried rice": "rice",
    "king oyster mushroom": "mushroom",
    "oyster mushroom": "mushroom",
    "enoki mushroom": "mushroom",
```

```
    "white button mushroom": "mushroom",
    "shiitake": "mushroom",
    "crab": "seafood",
    "shellfish": "seafood",
    "shrimp": "seafood",
    "peanut": "nut",
    "almond": "nut",
    "cashew": "nut",
    "walnut": "nut",
    "almond": "nut",

    "ice cream": "dessert_snack",
    "cake": "dessert_snack",
    "biscuit": "dessert_snack",
    "chocolate": "dessert_snack",
    "candy": "dessert_snack",
    "pudding": "dessert_snack",
    "egg tart": "dessert_snack",
    "popcorn": "dessert_snack",

    "spring onion": "vegetable",
    "bean sprouts": "vegetable",
    "okra": "vegetable",
    "white radish": "vegetable",
    "eggplant": "vegetable",
    "snow peas": "vegetable",
    "bamboo shoots": "vegetable",
    "garlic": "vegetable",
    "ginger": "vegetable",
    "cilantro mint": "vegetable", # Có thể gom v

}

# Exclude noisy auto-similarity pairs from merge
EXCLUDED_SIMILARITY_PAIRS = {
    tuple(sorted(("grape", "rape"))),
}

# Optional: only suggest similarity pairs that c
# Keep as 0 to disable this filter.
```

```
MIN_COMMON_IMAGES_FOR_SUGGESTION = 0

# Export controls.
DROP_IMAGES_WITH_NO_OBJECTS = True
COPY_ONLY_IMAGES_WITH_ANNOTATION = True
MAX_FILES_PER_SPLIT = None  # Use an integer for
EXECUTE_EXPORT = True      # Set True when ready

print("Configuration loaded.")
```

```
Configuration loaded.
```

In [12]:

```
# Resolve dataset root and load metadata
DATASET_ROOT = None
for p in CANDIDATE_ROOTS:
    if p.exists() and p.is_dir():
        DATASET_ROOT = p.resolve()
        break

if DATASET_ROOT is None:
    raise FileNotFoundError("Dataset root not fo

META_PATH = DATASET_ROOT / "meta.json"
if not META_PATH.exists():
    raise FileNotFoundError(f"meta.json not foun

with open(META_PATH, "r", encoding="utf-8") as f
    meta = json.load(f)

meta_classes = meta.get("classes", [])
id_to_title = {int(c["id"]): str(c["title"]).str
title_to_meta = {str(c.get("title", "")).strip(

print(f"DATASET_ROOT: {DATASET_ROOT}")
print(f"Original classes in meta: {len(meta_clas
for split in SPLITS:
    img_ok = (DATASET_ROOT / split / "img").exis
    ann_ok = (DATASET_ROOT / split / "ann").exis
    print(f"[{split}] img: {img_ok} | ann: {ann_
```

```
DATASET_ROOT: D:\CODE\PROJECT-CV\test\foodseg103
Original classes in meta: 103
[train] img: True | ann: True
[test] img: True | ann: True
```

In [13]:

```
def extract_class_name(obj: dict) -> str:
    title = str(obj.get("classTitle", "")).stri
    if title:
        return title

    class_id = obj.get("classId", None)
    try:
        class_id = int(class_id)
    except (TypeError, ValueError):
        return "unknown"

    return id_to_title.get(class_id, "unknown")


object_counter = Counter()
image_counter = Counter()
scan_rows = []

for split in SPLITS:
    img_dir = DATASET_ROOT / split / "img"
    ann_dir = DATASET_ROOT / split / "ann"

    image_files = sorted([p for p in img_dir.ite
    if MAX_FILES_PER_SPLIT is not None:
        image_files = image_files[:MAX_FILES_PE

    for img_path in tqdm(image_files, desc=f"Sc
        ann_path = ann_dir / f"{img_path.name}.
        status = "ok"
        num_objects = 0
        classes_in_img = set()

        if not ann_path.exists():
            status = "missing_annotation"
        else:
```

```
            try:
                with open(ann_path, "r", encodi
                    ann = json.load(f)

                objects = ann.get("objects", []
                num_objects = int(len(objects))

                for obj in objects:
                    cls = extract_class_name(obj
                    object_counter[cls] += 1
                    classes_in_img.add(cls)

                for cls in classes_in_img:
                    image_counter[cls] += 1
            except Exception:
                status = "bad_annotation_json"

        scan_rows.append({
            "split": split,
            "image_file": img_path.name,
            "image_path": str(img_path),
            "ann_path": str(ann_path),
            "status": status,
            "num_objects": num_objects,
        })

scan_df = pd.DataFrame(scan_rows)
class_stats_df = pd.DataFrame({
    "class_name": sorted(set(object_counter.keys
})
class_stats_df["objects"] = class_stats_df["clas
class_stats_df["images"] = class_stats_df["class
class_stats_df = class_stats_df.sort_values(["ob

print("Scan status by split:")
display(scan_df.groupby(["split", "status"]).si
print(f"Observed classes: {len(class_stats_df)}"
display(class_stats_df.head(20))
```

```
Scanning train:   0%|          | 0/3982 [00:00<?, ?it/s]
Scanning test:   0%|          | 0/2135 [00:00<?, ?it/s]
Scan status by split:
```

	split	status	count
0	test	ok	2135
1	train	ok	3982

```
Observed classes: 103
```

	class_name	objects	images
0	bread	1259	1259
1	carrot	1172	1172
2	chicken duck	1124	1124
3	sauce	1051	1051
4	tomato	1035	1035
5	potato	1001	1001
6	steak	983	983
7	broccoli	935	935
8	ice cream	790	790
9	cilantro mint	769	769
10	rice	611	611
11	pork	589	589
12	lemon	516	516
13	strawberry	514	514
14	lettuce	513	513
15	pie	494	494
16	cucumber	492	492
17	onion	458	458
18	corn	454	454
19	cake	387	387

In [19]:

```
# Rare class report + merge suggestions
SIMILARITY_THRESHOLD = 0.88

rare_df = class_stats_df[
    (class_stats_df["images"] < MIN_IMAGES_PER_(
    | (class_stats_df["objects"] < MIN_OBJECTS_I
].copy()

def build_class_to_images(scan_df):
    class_to_images = defaultdict(set)
    ok_rows = scan_df[scan_df["status"] == "ok"]
    for _, row in tqdm(ok_rows.iterrows(), total
        ann_path = Path(row["ann_path"])
        image_token = f"{row['split']}/{row['ima
        try:
            with open(ann_path, "r", encoding="u
                ann = json.load(f)
        except Exception:
            continue

        classes_in_img = set()
        for obj in ann.get("objects", []):
            cls_name = extract_class_name(obj)
            classes_in_img.add(cls_name)

        for cls_name in classes_in_img:
            class_to_images[cls_name].add(image_

    return class_to_images


def suggest_similar_pairs(
    names,
    class_to_images,
    min_ratio=0.88,
    excluded_pairs=None,
    min_common_images=0,
):
    names = sorted(set(names))
```

```
    excluded_pairs = excluded_pairs or set()
    rows = []
    for i in range(len(names)):
        for j in range(i + 1, len(names)):
            a, b = names[i], names[j]
            pair_key = tuple(sorted((a, b)))
            if pair_key in excluded_pairs:
                continue

            ratio = SequenceMatcher(None, a.lowe
            if ratio < min_ratio:
                continue

            imgs_a = sorted(class_to_images.get
            imgs_b = sorted(class_to_images.get
            common = sorted(set(imgs_a) & set(ir
            if len(common) < min_common_images:
                continue

            rows.append({
                "class_a": a,
                "class_b": b,
                "similarity": round(ratio, 3),
                "images_a": len(imgs_a),
                "images_b": len(imgs_b),
                "images_common": len(common),
                "example_image_a": imgs_a[0] if
                "example_image_b": imgs_b[0] if
                "example_image_common": common[0
            })

    if not rows:
        return pd.DataFrame(columns=[
            "class_a", "class_b", "similarity",
            "images_a", "images_b", "images_comm
            "example_image_a", "example_image_b"
        ])

    return pd.DataFrame(rows).sort_values(["sim
```

```
class_to_images = build_class_to_images(scan_df
similar_df = suggest_similar_pairs(
    class_stats_df["class_name"].tolist(),
    class_to_images=class_to_images,
    min_ratio=SIMILARITY_THRESHOLD,
    excluded_pairs=EXCLUDED_SIMILARITY_PAIRS,
    min_common_images=MIN_COMMON_IMAGES_FOR_SUG
)

print("Potentially rare classes (by current thre
display(rare_df.sort_values(["images", "objects"

print("Potential merge candidates with image pro
display(similar_df.head(50))
print("Check example_image_common first when val
print(f"Excluded similarity pairs: {len(EXCLUDED
```

```
Index class-image map:   0%|          | 0/6117
[00:00<?, ?it/s]
Potentially rare classes (by current threshold
s):
```

class_name	objects	images

```
Potential merge candidates with image provenanc
e:
```

class_a	class_b	similarity	images_a	images_b	ima

```
Check example_image_common first when validating
a merge decision.
Excluded similarity pairs: 1
```

In [20]:

```
# Build rebalance plan from thresholds + merge m
clean_merge_map = {
    str(k).strip(): str(v).strip()
    for k, v in MERGE_MAP.items()
    if str(k).strip() and str(v).strip()
```

```
}

known_classes = set(class_stats_df["class_name"
unknown_merge_keys = sorted(set(clean_merge_map
if unknown_merge_keys:
    print("Warning: some MERGE_MAP keys not four
    print(unknown_merge_keys[:20])

source_to_target = {c: clean_merge_map.get(c, c
target_to_sources = defaultdict(list)
for src, tgt in source_to_target.items():
    target_to_sources[tgt].append(src)

target_rows = []
for tgt, src_list in target_to_sources.items():
    objects = int(class_stats_df[class_stats_df
    images = int(class_stats_df[class_stats_df[
    target_rows.append({
        "target_class": tgt,
        "n_source_classes": len(src_list),
        "objects": objects,
        "images": images,
        "source_classes": ", ".join(sorted(src_
    })

target_stats_df = pd.DataFrame(target_rows).sor

kept_targets = set(target_stats_df.loc[
    (target_stats_df["images"] >= MIN_IMAGES_PE
    & (target_stats_df["objects"] >= MIN_OBJECT
    "target_class",
].tolist())

kept_targets |= {clean_merge_map.get(c, c) for
kept_targets -= {clean_merge_map.get(c, c) for

plan_rows = []
for src in sorted(known_classes):
    src_obj = int(class_stats_df.loc[class_stat
    src_img = int(class_stats_df.loc[class_stat
```

```
    tgt = source_to_target[src]
    tgt_obj = int(target_stats_df.loc[target_sta
    tgt_img = int(target_stats_df.loc[target_sta
    keep = tgt in kept_targets
    action = "merge_keep" if (src != tgt and kee
    plan_rows.append({
        "source_class": src,
        "target_class": tgt,
        "source_objects": src_obj,
        "source_images": src_img,
        "target_objects": tgt_obj,
        "target_images": tgt_img,
        "keep": keep,
        "action": action,
    })

plan_df = pd.DataFrame(plan_rows).sort_values(['

new_class_titles = sorted(kept_targets)
new_class_id_map = {name: i + 1 for i, name in 

print(f"Target classes after merge/filter: {len
print(f"Dropped source classes: {(~plan_df['keep
display(target_stats_df.head(30))
display(plan_df.head(50))
```

```
Target classes after merge/filter: 76
Dropped source classes: 9
```

	target_class	n_source_classes	objects	images	s
0	dessert_snack	8	1487	1487	
1	bread	1	1259	1259	
2	carrot	1	1172	1172	
3	vegetable	10	1159	1159	
4	chicken duck	1	1124	1124	
5	sauce	1	1051	1051	
6	tomato	1	1035	1035	
7	potato	1	1001	1001	
8	steak	1	983	983	
9	broccoli	1	935	935	
10	rice	1	611	611	
11	pork	1	589	589	
12	lemon	1	516	516	
13	strawberry	1	514	514	
14	lettuce	1	513	513	
15	pie	1	494	494	
16	cucumber	1	492	492	

	target_class	n_source_classes	objects	images	s
17	onion	1	458	458	
18	corn	1	454	454	
19	pepper	1	381	381	
20	french beans	1	333	333	
21	cheese butter	1	333	333	
22	fish	1	316	316	
23	egg	1	258	258	
24	mushroom	5	238	238	
25	asparagus	1	233	233	
26	orange	1	222	222	
27	french fries	1	222	222	
28	fried meat	1	222	222	
29	noodles	1	217	217	

	source_class	target_class	source_objects	source_i
0	bread	bread	1259	
1	carrot	carrot	1172	
2	cilantro mint	vegetable	769	
3	spring onion	vegetable	153	
4	garlic	vegetable	68	
5	snow peas	vegetable	56	
6	white radish	vegetable	40	
7	bean sprouts	vegetable	27	
8	ginger	vegetable	21	
9	eggplant	vegetable	17	
10	okra	vegetable	5	
11	bamboo shoots	vegetable	3	
12	chicken duck	chicken duck	1124	
13	sauce	sauce	1051	
14	tomato	tomato	1035	
15	potato	potato	1001	
16	steak	steak	983	
17	broccoli	broccoli	935	
18	rice	rice	611	
19	pork	pork	589	

	source_class	target_class	source_objects	source_i
20	lemon	lemon	516	
21	strawberry	strawberry	514	
22	lettuce	lettuce	513	
23	pie	pie	494	
24	cucumber	cucumber	492	
25	onion	onion	458	
26	corn	corn	454	
27	pepper	pepper	381	
28	cheese butter	cheese butter	333	
29	french beans	french beans	333	
30	fish	fish	316	
31	egg	egg	258	
32	white button mushroom	mushroom	119	
33	shiitake	mushroom	104	
34	king oyster mushroom	mushroom	7	
35	oyster mushroom	mushroom	5	
36	enoki mushroom	mushroom	3	
37	asparagus	asparagus	233	

	source_class	target_class	source_objects	source_i
38	french fries	french fries	222	
39	fried meat	fried meat	222	
40	orange	orange	222	
41	noodles	noodles	217	
42	cauliflower	cauliflower	213	
43	celery stick	celery stick	209	
44	sausage	sausage	205	
45	other ingredients	other ingredients	198	
46	green beans	green beans	192	
47	blueberry	blueberry	190	
48	cherry	cherry	183	
49	coffee	coffee	156	

In [21]:

```
# Export filtered + merged dataset
if len(new_class_titles) == 0:
    raise RuntimeError("No classes kept. Relax t

if EXECUTE_EXPORT:
    OUTPUT_ROOT.mkdir(parents=True, exist_ok=Tru

export_stats = []

for split in SPLITS:
    in_img_dir = DATASET_ROOT / split / "img"
    in_ann_dir = DATASET_ROOT / split / "ann"
```

```
    out_img_dir = OUTPUT_ROOT / split / "img"
    out_ann_dir = OUTPUT_ROOT / split / "ann"
    if EXECUTE_EXPORT:
        out_img_dir.mkdir(parents=True, exist_ol
        out_ann_dir.mkdir(parents=True, exist_ol

    image_files = sorted([p for p in in_img_dir
    if MAX_FILES_PER_SPLIT is not None:
        image_files = image_files[:MAX_FILES_PEI

    input_images = 0
    output_images = 0
    missing_ann = 0
    bad_ann = 0
    dropped_empty = 0
    input_objects = 0
    output_objects = 0

    for img_path in tqdm(image_files, desc=f"Exp
        input_images += 1
        ann_path = in_ann_dir / f"{img_path.name

        if not ann_path.exists():
            missing_ann += 1
            if not COPY_ONLY_IMAGES_WITH_ANNOTAT
                shutil.copy2(img_path, out_img_d
                output_images += 1
            continue

        try:
            with open(ann_path, "r", encoding="u
                ann = json.load(f)
        except Exception:
            bad_ann += 1
            continue

        objs = ann.get("objects", [])
        input_objects += len(objs)
```

```
        new_objects = []
        for obj in objs:
            src = extract_class_name(obj)
            tgt = source_to_target.get(src, src
            if tgt not in new_class_id_map:
                continue

            obj_new = dict(obj)
            obj_new["classTitle"] = tgt
            obj_new["classId"] = int(new_class_
            new_objects.append(obj_new)

        output_objects += len(new_objects)

        if DROP_IMAGES_WITH_NO_OBJECTS and len(
            dropped_empty += 1
            continue

        if EXECUTE_EXPORT:
            ann_new = dict(ann)
            ann_new["objects"] = new_objects

            shutil.copy2(img_path, out_img_dir /
            with open(out_ann_dir / f"{img_path
                json.dump(ann_new, f, ensure_as

        output_images += 1

    export_stats.append({
        "split": split,
        "input_images": input_images,
        "output_images": output_images,
        "missing_annotation": missing_ann,
        "bad_annotation": bad_ann,
        "dropped_empty_images": dropped_empty,
        "input_objects": input_objects,
        "output_objects": output_objects,
    })

export_df = pd.DataFrame(export_stats)
```

```
display(export_df)

if EXECUTE_EXPORT:
    # Build new meta.json
    new_meta_classes = []
    for class_name in new_class_titles:
        src_candidates = sorted(target_to_source
        base = None
        for src in src_candidates:
            if src in title_to_meta:
                base = dict(title_to_meta[src])
                break

        if base is None:
            base = {
                "title": class_name,
                "shape": "bitmap",
                "color": "#FFFFFF",
                "geometry_config": {},
                "hotkey": "",
            }

        base["title"] = class_name
        base["id"] = int(new_class_id_map[class_
        new_meta_classes.append(base)

    new_meta = dict(meta)
    new_meta["classes"] = new_meta_classes

    with open(OUTPUT_ROOT / "meta.json", "w", en
        json.dump(new_meta, f, indent=2, ensure_

    # Save mapping and plan for reproducibility
    plan_out = OUTPUT_ROOT / "class_rebalance_pl
    plan_df.to_csv(plan_out, index=False)

    merged_rows = []
    for src in sorted(source_to_target):
        tgt = source_to_target[src]
        merged_rows.append({
```

```
            "source_class": src,
            "target_class": tgt,
            "kept": tgt in new_class_id_map,
            "new_class_id": new_class_id_map.get
        })

    mapping_df = pd.DataFrame(merged_rows)
    mapping_df.to_csv(OUTPUT_ROOT / "class_mapp

    print(f"Export completed: {OUTPUT_ROOT.reso
    print(f"Saved: {plan_out.resolve()}")
    print(f"Saved: {(OUTPUT_ROOT / 'class_mappi
else:
    print("Dry run mode: set EXECUTE_EXPORT = Tr
```

```
Export train:   0%|          | 0/3982 [00:00<?,
?it/s]
Export test:   0%|          | 0/2135 [00:00<?, ?
it/s]
```

	split	input_images	output_images	missing_annota
0	train	3982	3963	
1	test	2135	2122	

```
Export completed: D:\CODE\PROJECT-CV\test\foodse
g103_rebalanced
Saved: D:\CODE\PROJECT-CV\test\foodseg103_rebala
nced\class_rebalance_plan.csv
Saved: D:\CODE\PROJECT-CV\test\foodseg103_rebala
nced\class_mapping.csv
```

In [22]:

```
# Quick sanity checks after export
if EXECUTE_EXPORT:
    out_meta = OUTPUT_ROOT / "meta.json"
    print(f"meta.json exists: {out_meta.exists(

    for split in SPLITS:
```

```
        out_img = OUTPUT_ROOT / split / "img"
        out_ann = OUTPUT_ROOT / split / "ann"
        n_img = len([p for p in out_img.iterdir
        n_ann = len(list(out_ann.glob("*.json")
        print(f"[{split}] images={n_img}, annota

    print("Sanity check complete.")
else:
    print("No files were exported yet. Enable E
```

```
meta.json exists: True
[train] images=3982, annotations=3982
[test] images=2135, annotations=2135
Sanity check complete.
```

In [ ]:

```
%pip install "qrcode[pil]"
```

Failed to start the Kernel.

Unable to start Kernel 'base (Python 3.13.11)' due to a timeout waiting for the ports to get used.

View Jupyter <a href='command:jupyter.viewOutput'>log</a> for further details.

In [ ]:

```
import qrcode

# The link you want to turn into a QR code
link = "https://www.facebook.com/profile.php?id=

# Generate the QR code
img = qrcode.make(link)

# Save the generated image
img.save("my_link_qr.png")

print("QR Code generated successfully!")
```